

Article

Detecting Ransomware Through Dynamic API Call Monitoring and Machine Learning

Anas AlMajali ^{1,*}, Waleed Dweik ^{2,3}, Yahia Abdelghafur ³, Obada Alhasan ³, Omar Tawalbeh ³ and Moustafa Amer ³

¹ Department of Computer Engineering, Faculty of Engineering, The Hashemite University, Zarqa 13133, Jordan

² Computer Engineering Department, University of Jordan, Amman 11942, Jordan; wdweik@aus.edu

³ Department of Computer Science and Engineering, American University of Sharjah, Sharjah 26666, United Arab Emirates; b00092768@aus.edu (Y.A.); b00091720@aus.edu (O.A.); b00091148@aus.edu (O.T.); b00093054@aus.edu (M.A.)

* Correspondence: almajali@hu.edu.jo

Abstract

Ransomware is malicious software that encrypts user files, rendering them inaccessible unless a ransom is paid to the attacker. Ransomware detection has been a challenge recently. In this paper, we present a comprehensive analysis of ransomware detection using machine learning (ML) on Application Program Interface (API) calls made by malicious and non-malicious processes. The analysis is conducted on different ML models, including Random Forest, Long Short-Term Memory, Support Vector Machine, K-Nearest Neighbors, XGBoost, Linear Discriminant Analysis, and an ensemble model. The models were analyzed using two datasets: a primary public dataset and a zero-day dataset representing previously unseen real-world ransomware generated with no overlap with the primary dataset. For the primary dataset, the results indicate high detection accuracy of 99.82% for the ensemble model. On the other hand, testing on the zero-day ransomware results in a 96.3% accuracy while imposing minimal overhead on the CPU and memory of the machine. To boost the confidence in the detection method, we analyzed the detection time and the percentage of files saved because of the timely response of the proposed detection method. The results indicate that detection was done in a timely manner, saving on average 93% of the files under ransomware attack.

Keywords: cybersecurity; machine learning; malware; ransomware; ransomware detection; sandbox



Academic Editor: Douglas O'Shaughnessy

Received: 27 March 2026

Revised: 18 May 2026

Accepted: 20 May 2026

Published: 23 May 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

Ransomware is a type of malicious software (malware) that prevents legitimate users from accessing their computer files. Computer systems can be infected by ransomware through any of the traditional malware infection mechanisms, such as software vulnerabilities, phishing emails, drive-by downloads, and trojan horses.

Crypto-ransomware is a type of ransomware that encrypts user files using a strong encryption algorithm, making them unusable without the decryption keys. This is the most common type of ransomware. WannaCry is a famous crypto-ransomware that infected more than 200,000 computers running the vulnerable Microsoft Windows operating system in 2017 [1]. There are other types of ransomware, such as locker ransomware, which locks out the user from the computer, rendering it inaccessible; an example is Locky [2].

Scareware ransomware, which tries to deceive the user with pop-up messages demanding payment in exchange for a fake fix (e.g., rogue antivirus products) or law violation fine.

Generally, there are two main methods that are used for ransomware detection: signature-based and behavior-based. In signature-based detection, a unique signature is created for the malicious file containing the ransomware. Security software like antivirus software scans for instances of the ransomware identified by the signatures. The security software has to be regularly updated for new malware signatures. This method is vulnerable to zero-day attacks for which no signatures are available. On the other hand, behavior-based methods rely on analyzing the behavior of the ransomware after it starts execution and raising a flag once a suspicious behavior is detected. For example, rapid file changes in the system might indicate the presence of a ransomware encrypting the files (i.e., changing them). Another example of detection based on the behavior is the use of the sequence of events (e.g., API calls) made by a ransomware.

In this paper, we utilize and optimize machine learning (ML) techniques to detect ransomware following a behavior-based method. Our main method relies on an ensemble model combining XGBoost, Support Vector Machine (SVM), and Long Short-Term Memory (LSTM) to classify a process based on behavior as ransomware or benign. The behavior features are extracted from the API call sequences generated by the running processes. The main contributions of this paper can be summarized as follows:

1. **Dataset construction:** The primary dataset was processed to address class imbalance and bias. Three actions were taken: incorporating a supplementary dataset that is more balanced, using the Synthetic Minority Over-sampling Technique (SMOTE) to increase the minority samples and undersampling the majority class by 40% (Section 3).
2. **Sensitivity Analysis:** A sensitivity analysis was performed on the number of sequential API calls that must be processed to accurately detect ransomware. As demonstrated by Figure 1, the optimal sequence length is 100 APIs (Section 3).
3. **A comprehensive analysis of the ransomware detection using different ML models,** namely, RF, LSTM, SVM, K-Nearest Neighbors (KNN), XGBoost, Linear Discriminant Analysis (LDA), and the ensemble model. The models were analyzed (i.e., training and testing) using two datasets: a primary dataset that is publicly available and a zero-day dataset that we created (Section 4).
4. **Evaluation protocol:** The evaluation of this work was done in two steps. The first was testing the ML models on the dataset. The second step was used to boost the confidence in the dataset result and was done on zero-day ransomware. The zero-day ransomware testing was done in real-time using 109 real ransomware strains that were carefully selected to avoid overlap with the primary dataset. Specifically, the hashes of the running samples were matched with those in the primary dataset to avoid overlap (Section 4).
5. **Proposed ML detection model:** An ensemble ML model combining XGBoost, SVM, and LSTM that achieved high testing accuracy on the primary dataset (99.82%) and the zero-day dataset (96.3%) while consuming minimal system resources (Section 5).

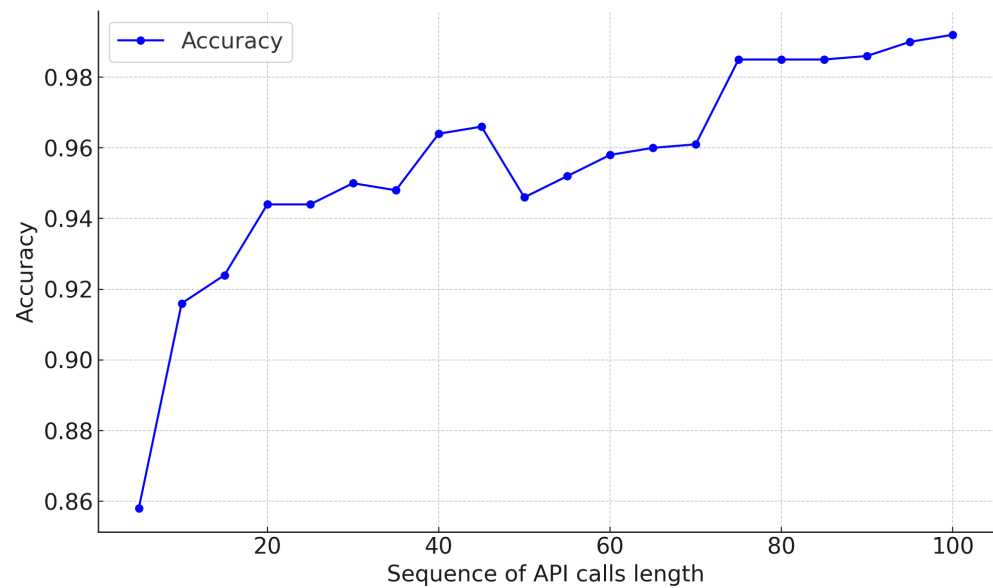


Figure 1. Classification accuracy versus API call sequence length from 5 to 100 API calls.

The remainder of this paper is organized as follows: Section 2 discusses infamous ransomware examples and current ransomware detection methods. Section 3 describes the datasets used to build the proposed ML model and the experimental setup. Section 4 presents the proposed methodology and model architecture. The experimental results and main findings are analyzed and discussed in Section 5. Finally, Section 6 concludes the paper and suggests future work.

2. Related Work

In this section, we provide a literature review of the most famous ransomware attacks and state-of-the-art ransomware detection methods.

2.1. Historical Overview of Ransomware

Ransomware can be broadly classified into locker and crypto-ransomware. Locker ransomware [3] prevents access to victim devices, while crypto ransomware [4] encrypts files on the victim devices, rendering them inaccessible until the ransom is paid. Moreover, ransomware can be classified according to their target scope into consumer-targeted [5] and enterprise-targeted [6]. Consumer-targeted ransomware attacks individual users' devices through malicious emails and downloads, whereas enterprise-targeted ransomware attacks the devices of large organizations using network-based techniques. There are various metrics to classify ransomware, such as propagation method and ransom demand strategy. In this subsection, we provide an overview of the most famous ransomware attacks from different classes over the years.

2.1.1. The AIDS Trojan (1989)

The first known ransomware encrypted filenames instead of the files themselves and required payment by mail order. The ransomware used basic symmetric encryption that stored its keys on the compromised system. Security experts managed to break the encryption system, which allowed them to restore all affected files. The physical distribution of the attack through floppy disks restricted its propagation. The attack caused an estimated \$189,000 in damages [5].

2.1.2. CryptoLocker (2013)

A highly destructive crypto-ransomware that spread primarily through malicious email attachments and leveraged the GameOver ZeuS botnet for distribution. It used AES-256 for file encryption and RSA-2048 for securing the decryption keys, ensuring that victims could not recover files without paying. To avoid detection, the malware disguised itself as a legitimate Windows update process. Unlike earlier ransomware, CryptoLocker stored the private decryption keys exclusively on attacker-controlled command-and-control (C2) servers, making brute-force decryption impossible. In 2014, a coordinated operation by the FBI and cybersecurity researchers dismantled the GameOver ZeuS botnet, disrupting CryptoLocker's infrastructure. The security firm FireEye later released a free decryption tool by recovering RSA keys from seized C2 servers. The ransomware infected more than 500,000 machines around the world, extorting an estimated \$3 million in ransom payments before its takedown [4].

2.1.3. CryptoWall (2014)

A successor to CryptoLocker, improving file obfuscation, persistence mechanisms, and C2 communication. It differed from its predecessor by spreading primarily through malicious advertising (malvertising) and phishing emails. Moreover, the ransomware renamed encrypted files and deleted volume shadow copies to prevent recovery. The exact number of infections is unknown, but there were global campaigns that targeted thousands of users. The FBI estimated that victims paid more than \$18 million in ransom payments before mitigation efforts took hold [6].

2.1.4. TeslaCrypt (2015)

A ransomware strain that used AES-128 in Cipher Block Chaining (CBC) mode for file encryption and Elliptic Curve Diffie-Hellman (ECDH) to secure the encryption keys, making it particularly resilient against decryption attempts. Initially targeting documents and personal files, TeslaCrypt evolved to remove local key storage in later versions, forcing victims to interact with attacker-controlled C2 servers for any chance of recovery. In a surprising turn of events, the ransomware's operators released a master decryption key in May 2016, enabling security researchers to develop a free decryption tool for affected users. Prior to that, TeslaCrypt had infected more than 32,000 systems, with ransom demands ranging from \$250 to \$1000 per victim [7].

2.1.5. Locky (2016)

Distributes through phishing emails containing malicious Word macros. Upon infection, it encrypted files and appended a specific extension, specifically targeting businesses and healthcare institutions. The ransomware employed a dual-encryption scheme: AES-128 for file encryption and RSA-2048 to secure the keys, which were stored exclusively on attacker-controlled C2 servers, complicating recovery efforts. While full decryption without payment was nearly impossible, some partial file recovery was achieved through forensic memory analysis. Locky's impact was widespread, with multiple large-scale infection waves affecting organizations globally. Among its most high-profile victims was Hollywood Presbyterian Medical Center, which faced severe operational disruptions forcing staff to revert to paper records and ultimately paid \$17,000 in ransom to restore systems. The financial and operational toll on hospitals and businesses underscored Locky's notoriety as one of the most damaging ransomware campaigns of its time [3].

2.1.6. Jigsaw (2016)

In 2020, Jigsaw ransomware took a unique approach by not only encrypting files but also deleting them at regular intervals if the ransom was not paid. It used AES encryption with a major flaw: the code contained a fixed key that could actually be used to decrypt it. The ransomware was designed to erase files every hour until the ransom was paid and mainly affected common file extensions such as *.doc, *.txt, *.jpg and *.png. Typically, it would show an image of the Jigsaw movie villain to threaten the victims. Since the decryption key was stored locally within the ransomware executable, a free decryption tool was created and made available for download shortly after its discovery. Jigsaw mainly hit hundreds of small businesses, especially their financial data, but the exact financial impact is reported to be only the ransom paid [7].

2.1.7. NotPetya (2016)

The ransomware emerged in 2016 as a destructive form of malware that went beyond standard file encryption by encrypting the entire hard drive and modifying the Master Boot Record (MBR), resulting in unbootable systems. The ransomware displayed a ransom note that demanded Bitcoin payment while encrypting the entire disk, including system boot files, through full disk encryption based on MBR modification. The malware used a modified AES encryption to lock down entire disk partitions but it did not store decryption keys or use a randomized key for each infection, which made recovery impossible. System behavior monitoring solutions could identify the fast and extensive encryption process and MBR/MFT structural modifications. NotPetya caused major damage to over 200,000 machines worldwide and Maersk suffered \$300 million in damages, contributing to a worldwide estimated loss of \$10 billion [8].

2.1.8. WannaCry (2017)

WannaCry stood out by spreading through the network using a worm-like mechanism, particularly through the exploitation of the SMB vulnerability known as Eternal-Blue (an SMBv1 flaw). This caused it to spread quickly across systems, locking files and demanding Bitcoin payments, mainly targeting businesses and organizations with networked computers. The ransomware used AES-128 to encrypt files and RSA-2048 to lock the encryption keys, splitting files into chunks to boost speed. Despite the intention of encrypting the key and sending it to the attacker's command-and-control (C2) servers, researchers managed to create tools such as "WannaKey" and "WannaKiwi" that could obtain decryption keys from the memory of compromised systems. The attack affected over 230,000 computers in 150 countries worldwide and led to an estimated \$4–8 billion in total damages [9].

2.1.9. SamSam (2017)

The SamSam ransomware was particularly notable for its manual deployment following brute-force attacks on weak passwords, and it was mainly used to compromise healthcare, government and corporate systems. This ransomware used AES-256 encryption, and all encryption keys were generated on the infected machines; unlike other ransomware, the keys were not sent to external servers, which made recovery quite challenging. In this regard, since the attacker's private key was not available, decryption was almost impossible, although no single tool for decryption existed, some amount of data could be recovered through memory forensic analysis. SamSam affected hundreds of organizations, including hospitals and government agencies, and caused more than \$30 million in damages to several entities [6].

2.1.10. GandCrab (2017)

GandCrab ransomware emerged in 2017 and was one of the most successful Ransomware as a Service (RaaS) because it used both file encryption and data stealing along with the Vidar malware to steal user credentials. This ransomware employed the Salsa20 stream cipher for the encryption process, which was faster than AES-based ransomware, and further secured the encryption key with RSA-2048. The encryption keys were stored on the attacker's Command and Control (C2) servers. In June 2019, a major breakthrough was made when security researchers and law enforcement got their hands on the master decryption keys, and a free decryptor was released. At its peak, it affected more than 50,000 victims monthly and collected more than \$2 billion in ransom payments [10].

2.1.11. Petrwrap (2017)

Petya ransomware functioned differently from the other variants of the software. It used Rijndael (an AES variant) to lock files and RSA encryption to lock the AES key. The victims were required to visit a Tor site to pay the ransom and download a decryption tool, and the malware affected both personal and business files. Petrwrap used AES with SHA-256 key derivation and RSA encryption for the key storage and, unlike other ransomware, encrypted the whole disk instead of individual files. The private decryption key was kept on the attacker's server, and some of the variants of the malware also altered the boot sector to prevent the victims from accessing their machines. Some of the variants were decryptable, however, the total damages associated with the Petya attacks were estimated to be in the hundreds of millions and thousands of machines were affected in 2017 [7].

2.1.12. Ryuk (2018)

It usually got access through the deployment of TrickBot or Emotet trojan malware. This ransomware implemented a strong AES-256 encryption in CBC mode for file locking and RSA-4096 for encryption key management, which made decryption nearly impossible. The encryption keys were stored on the attacker's C2 infrastructure remotely. As a result, no single universal decryption tool was developed to help victims recover the files, however, some of the affected parties were able to restore their data through complex forensic analysis of the infected computers' memory. It is estimated that Ryuk infected over 100,000 computers worldwide and is remembered for its attacks on critical infrastructure such as newspaper printing plants and municipal services and caused enormous financial losses. It is reported that Ryuk victims paid out about \$150 million in ransoms [6].

2.1.13. Cerber (2021)

Through its RaaS model, the attackers encrypted system files then requested 1 Bitcoin (approximately \$590 in 2016) before providing decryption keys. AES encryption locked files so they became unusable until the attackers provided the decryption key. The attackers managed remote server storage for essential keys that blocked local decryption unless the attackers provided the decryption key. Research identified a potential detection system that used machine learning algorithms alongside Software-Defined Networking and Programmable Forwarding Engines to analyze HTTPS traffic flows. During July 2016, Cerber infected 150,000 devices in 201 countries, which led to an estimated \$195,000 in ransom payments in that month, suggesting annual revenue potential of \$2.3 million for the cybercriminals behind it [11].

Table 1 provides a summary of ransomware descriptions, encryption method, and encryption key storage. On the other hand, Table 2 provides a summary of the recovery method, number of infected devices, and the cost of damage. As this review demonstrates, it is becoming harder to decrypt files once encrypted by ransomware. Attackers are inventing

new ways to conceal the encryption keys, rendering decryption almost impossible [5,12]. As a result, it is important to focus on ransomware detection and prevention measures.

Table 1. Ransomware, description, encryption method, and key storage.

Ransomware	Description	Encryption Method	Key Storage
Aids Trojan (1989) [5]	First known ransomware. Encrypted filenames instead of files themselves. Demanded payment via mail order.	Simple symmetric encryption	Keys were stored locally on the infected machine
CryptoLocker (2013) [4]	Spreads via email and Gameover Zeus botnet.	RSA-2048 + AES-256	Stored on C2 servers
CryptoWall (2014) [6]	Uses malvertising/phishing; successor to CryptoLocker.	AES-256 + RSA-2048	Remotely stored
TeslaCrypt (2015) [7]	Targets personal data; strong ECC + AES.	AES-128 + ECDH	Stored on C2 servers
Locky (2016) [3]	Delivered via Word macro phishing.	RSA-2048 + AES-128	Stored on remote servers
Jigsaw (2016) [7]	Deletes files hourly if unpaid.	AES (hard-coded)	Local in executable
NotPetya (2016) [8]	MBR modifier, full disk encryption.	AES (disk-level)	Key not stored
WannaCry (2017) [9]	Worm-like; uses EternalBlue SMBv1.	AES-128 + RSA-2048	Sent to attacker via C2
SamSam (2017) [6]	Brute-force entry; manually deployed.	AES-256	Locally generated keys
GandCrab (2017) [10]	RaaS; uses Vidar malware too.	Salsa20 + RSA-2048	Stored on C2 servers
Petrwrap (2017) [7]	Petya variant with full-disk encryption.	AES + RSA + SHA-256	Stored on C2
Ryuk (2018) [6]	Targets enterprises via TrickBot.	AES-256 + RSA-4096	Remote attacker-controlled servers
Cerber (2021) [11]	RaaS; demanded 1 Bitcoin in 2016.	AES-based	Remote attacker server

Table 2. Impact, recovery, and references for ransomware cases.

Ransomware	Recovery Method	Number of Infected Machines	Cost of Damage
Aids Trojan (1989) [5]	Cracked by security researchers.	Spread via floppy disks.	\$189,000
CryptoLocker (2013) [4]	FBI took down botnet in 2014.	500,000+ infections	\$3 million
CryptoWall (2014) [6]	Detected via file monitoring.	Unknown; widespread.	\$18 million
TeslaCrypt (2015) [7]	Master key released in 2016.	32,000+ machines	\$250–1000/victim
Locky (2016) [3]	Partially recoverable via memory forensics.	Unknown	\$17,000/incident
Jigsaw (2016) [7]	Free decryptor released early.	Hundreds of SMBs	Unknown
NotPetya (2016) [8]	Detection via MBR/MFT behavior.	200,000+ globally	\$10 billion
WannaCry (2017) [9]	WannaKiwi/WannaKey tools available.	230,000+ in 150 countries	\$4–8 billion
SamSam (2017) [6]	No universal decryptor. Some memory dumps helped.	Hundreds of orgs.	\$30+ million
GandCrab (2017) [10]	Master decryptor released in 2019.	50,000+/mo. peak	\$2+ billion
Petrwrap (2017) [7]	Some variants recoverable.	Thousands globally	Hundreds of millions
Ryuk (2018) [6]	Partial recovery with memory dumps.	100,000+ estimated	\$150 million
Cerber (2021) [11]	Detected via flow-based ML on SDN.	150,000 devices (July 2016)	\$2.3 million

2.2. Historical Overview of Ransomware Detection Techniques

Similar to any malware, ransomware detection methods can be categorized into signature-based detection and behavioral-based detection. Signature-based detection relies on finding signatures in the ransomware executable files and accordingly flagging the file as ransomware. While effective against known ransomware, this method is not effective against zero-day attacks. Many methods were used in the literature to identify a signature for the ransomware that include hashing the executable file, similarity hashing, and analyzing API calls within the executable file [4,13–15].

On the other hand, a lot of the work in the literature has focused on behavioral-based detection with the main goal of detecting zero-day attacks [11,16–23]. The main disadvantage of such methods is that the ransomware has to start running to be detected, risking the encryption of some files. Hence, behavior-based detection methods compete to achieve the high detection accuracy and the short detection time, which are the main goals of this paper.

Chen et al. [16] proposed a method for ransomware detection using Term Frequency-Inverse Document Frequency (TF-IDF) for feature extraction, Fisher's LDA for dimensionality reduction, and Extremely Randomized Trees (Extra Trees) for classification. The detection method relies on finding the most discriminating features for seven ransomware families. Crypto-API calls, drop and execute, and unzipping files are examples of these

discriminating features. This method achieved a range of accuracy between 91.8% and 99.9% for the seven ransomware families.

In [13], the authors introduced a detection system that distinguishes between benign and ransomware samples using 206 common API calls. Three ML models were used for the evaluation with KNN algorithm providing the highest accuracy of 99%.

Naik et al. [14] proposed a two-stage detection method combining fuzzy hashing and clustering to categorize ransomware samples. Addressing the challenges posed by polymorphic ransomware and frequent versioning, the authors used SSDEEP and SDHASH fuzzy hashing methods to generate similarity scores between samples and then applied clustering methods like k-means and CLARA to group related samples. Utilizing static ransomware detection based on similarity detection, SDHASH identified a higher number of matches (108 out of 112) compared to SSDEEP (104 out of 112), though SSDEEP produced stronger, more reliable similarity scores.

The authors in [17] presented a machine learning-based ransomware detection approach that leverages low-level memory access patterns collected through a light hypervisor to bypass traditional operating system vulnerabilities. The authors developed a BitVisor-based hypervisor that monitors memory access patterns using Extended Page Table (EPT) violations, creating an additional protection layer beneath conventional OS-based security mechanisms. The system collected memory access patterns from eight classes: three ransomware samples (WannaCry, Sodinokibi/REvil, and Darkside), one wiper malware (CaddyWiper), and four benign applications (Idle, AESCrypt, Zip compression, and Office suite with web browser). Classification performance was evaluated using Random Forest, Support Vector Machine, and KNN models under 10-fold cross-validation. The Random Forest classifier achieved the best results with an F-1 score of 0.93 for eight-class classification and 0.95 for binary classification (malicious vs. benign), demonstrating 95% effectiveness in distinguishing ransomware and wiper malware from legitimate applications.

Ayub et al. [15] proposed RWArmor, a hybrid detection system that combines static and dynamic analysis using a Random Forest classifier. Targeting active cryptographic Windows ransomware, the method achieved 91.75% accuracy, a 7.09% false-positive rate (FPR), and a 9.53% false-negative rate (FNR). It also reported 91.99% precision, 90.47% recall, and an F-Score of 91.2%. While the accuracy is slightly lower than some other approaches, the balance between precision and recall indicates reliable detection with manageable error rates. The authors in [24] proposed Zero-Ran Sniff (ZRS), an approach combining zero-shot learning with static analysis of Portable Executable (PE) files' headers targeting zero-day ransomware. The method achieved 96.02% accuracy, a 5.46% false-positive rate (FPR), and a 1.53% false-negative rate (FNR). It also recorded 91.49% precision, 98.47% recall, and an F-Score of 94.85. The high recall and F1-Score suggest strong detection capabilities even for previously unseen ransomware variants.

Kok et al. [18] introduced a pre-encryption detection algorithm that combines signature based detection and behavioral based detection. Signature based detection uses SHA-256 to compare file signatures. On the other hand, pre-encryption APIs are used with ML techniques to detect ransomware dynamically. Tested on Cryptowall ransomware, the system achieved 99.07% accuracy, a 1% false-positive rate (FPR), and 0% false-negative rate (FNR). It also maintained 99% precision, 100% recall, and an F-Score of 99.5%.

Designed to combat Jigsaw ransomware, RWGuard monitors file system I/O operations and performs entropy checks to detect encryption in real-time [19]. The system achieved 96.55% accuracy, a 0.08% false-positive rate, and a 4% false-negative rate. It also reported 96% precision, 96% recall, and an F-Score of 96, demonstrating consistent performance with extremely low false alarms.

Bae et al. [19] used dynamic analysis and machine learning (Random Forest) to detect CryptoLocker ransomware. The model achieved 98.65% accuracy, a 3.58% false-positive rate, and a 0.65% false-negative rate. It also recorded 99.48% precision, 99.35% recall, and an F-Score of 99.41 [20].

Cusack et al. [11] proposed a flow-based detection system leveraging Software-Defined Networking (SDN) to analyze HTTPS traffic patterns between the victim machine and the C2 server. The approach achieved 94.4% accuracy, a 12.5% false-positive rate (FPR), and 0% false-negative rate (FNR). It also reported 90.91% precision, 100% recall, and an F-Score of 95.2. While the FPR is relatively high, the zero FNR ensures no ransomware evades detection, making it useful for high-security environments.

Cen et al. [21] proposed an early ransomware detection method called RansoGuard. RansoGuard detects ransomware by analyzing API calls that are made by ransomware before encryption. A Recurrent Neural Network (RNN) classifier was used to distinguish between ransomware and benign software. This method achieved a recall of 96.18% and an accuracy of 94.26%.

Finally, in [22,23], the authors use hashing methods to detect rapid changes in system and user files to flag a ransomware. The authors utilize similarity hashing modes of operation to perform fast and efficient file scanning improving the detection time. Table 3 summarizes the targeted ransomware families, detection methods, and various evaluation metrics for the studies discussed above.

Table 3. Summary of ransomware detection methods and metrics.

Research Paper Title	Ransomware Family	Detection Method	Accuracy	FPR	FNR	Precision	Recall	F1-Score
Cusack et al. [11]	Cerber	Flow-based + NetFlow Features	94.40	12.50	0.00	90.91	100.00	95.20
Chen et al. [16]	Multi-families	TF-IDF (values depend on the ransomware family)	91.80–99.90	0.00	0.00–69.20	100.00	30.80–99.70	47.09–99.85
Almousa et al. [13]	Petya	API-based + ML	99.18	1.54	1.00	99.00	99.00	99.00
Naik et al. [14]	WannaCry/ WannaCryptor	Static fuzzy hashing and clustering	108/112 similarity	-	-	-	-	-
Hirano et al. [17]	Sodinokibi/REvil, Dark-side	Memory access using hypervisor	-	-	-	-	-	0.95
Ayub et al. [15]	Active crypto Windows ransomware	Random Forest (static+dynamic)	91.75	7.09	9.53	91.99	90.47	91.22
Cen et al. [24]	TeslaCrypt	ZRS (zero-shot learning + sniffing)	96.02	5.46	1.53	91.49	98.47	94.85
Kok et al. [18]	Cryptowall	Pre-encryption detection (SHA-256)	99.07	1.00	0.00	99.00	100.00	99.50
Mehnaz et al. [19]	Jigsaw	File I/O Entropy Monitoring	96.55	0.08	4.00	96.00	96.00	96.00
Bae et al. [20]	Cryptlocker	Dynamic + Random Forest	98.65	3.58	0.65	99.48	99.35	99.41
Cen et al. [21]	Variation	Dynamic	94.26	-	-	96.92	96.18	96.55
AlMajali et al. [22,23]	Variation	Dynamic	100	-	-	-	-	-

3. DataSets and Experimental Setup

In this section, we describe how the dataset was collected and divided. We utilize two datasets: the primary and zero-day datasets. The primary dataset is used to train and test the various evaluated machine learning (ML) models as normally done with ML applications. The zero-day dataset is used for testing purposes only by mimicking the scenarios of zero-day ransomware attacks. Moreover, this section describes the experimental setup used to train, test, and contain the execution of zero-day ransomware.

3.1. Primary Dataset

This dataset contains a comprehensive set of dynamically generated Win32 API call sequences, each corresponding to either ransomware or benign process behavior. This dataset was sourced from [25] and originally consisted of 42,797 labeled ransomware samples and 1079 benign samples. In real-world scenarios, it is the opposite, where most processes are benign. Our preliminary results suffered from bias that led to low accuracy and high false positives. This is why we had to reduce the ransomware sample size (majority) and increase the benign sample size (minority) [26]. To address the substantial class imbalance and improve generalization performance, three actions were performed:

1. Incorporation of an additional 4000 labeled samples from a supplementary dataset published by Li et al. [27], which includes a more balanced distribution between benign and malicious samples.
2. Application of the Synthetic Minority Over-sampling Technique (SMOTE). This allowed us to oversample the minority class (benign samples) to ensure balanced training data, enhancing the model's sensitivity and specificity in detecting both benign and malicious processes.
3. Dropping 40% of ransomware samples, to make benign samples the majority class, like in real-world scenarios.

After these actions, the total size of the dataset becomes 68,475, out of which 42,797 samples are benign and 25,678 samples are ransomware. 80% of the samples (54,780) were used during training and 20% of the samples (13,695) were used for testing. We also want to acknowledge that explicit ransomware family labels are not available in the primary dataset.

The transformations made in this work may introduce potential bias. For example, SMOTE may cause over-generalization, whereas dropping 40% of the majority samples may lead to information loss. In addition, when incorporating a supplementary dataset, sampling or covariate shift may occur if the dataset was collected under different conditions. These factors highlight the importance of collecting realistic datasets from real-world environments.

Each sample is encoded as a sequential record of the first 100 API calls made by a process during execution. This fixed-length representation was determined after empirical experimentation. We tested multiple sequence lengths (from 5 to 100 API calls in steps of 5), and found that classification performance began to plateau after 80 API calls, as shown in Figure 1. Based on this analysis, we selected 100 as the standard sequence length, which captures sufficient behavioral context for early detection without introducing redundancy or overfitting.

3.2. Zero-Day Ransomware Testing Dataset

In addition to the primary dataset, we curated a separate set of 109 previously unseen ransomware binaries obtained from VirusShare [28], spanning recent and emerging ransomware families not included in the training corpus along with 191 benign samples. This dataset is collected to simulate zero-day attack scenarios. The selected samples were verified by ensuring that their hash signatures did not appear in any part of the training

dataset. Moreover, the ransomware families were chosen from those discovered and published significantly after the collection date of the primary dataset, thereby ensuring the novelty of the samples. However, we cannot rule out that some zero-day ransomware testing samples may belong to the same family as the training samples because family labels are not provided in the primary dataset. Capturing the API call sequences of the samples in this dataset is described in Section 4.2.

3.3. Experimental Setup

The experiments in this paper were conducted using a high-performance computing environment comprising an Intel(R) Core(TM) i7-13620H CPU, 32 GB of RAM, and a 500 GB SSD, running on Windows 11. This hardware configuration allowed us to efficiently train and evaluate multiple deep learning and machine learning models across large datasets without significant processing bottlenecks. In addition, this hardware setup was used to capture the API call sequences for the samples in the zero-day dataset.

We deployed a Cuckoo Sandbox instance within a virtualized Ubuntu 22.04 LTS environment. Inside this sandbox, a dedicated Windows 7 virtual machine was configured as the execution environment for ransomware samples. This nested virtualization strategy ensured containment and safety while maintaining realistic execution conditions. The guest machine mimicked a standard user environment with common file types and programs installed to encourage natural malware behavior.

API calls made by running processes were captured using Cuckoo's built-in behavior logging and parsing tools. These logs were then preprocessed into sequences suitable for model training and testing. The sandbox-based approach provided the necessary infrastructure for dynamic behavioral monitoring; this formed the foundation for our machine learning-based detection system.

4. Methodology

This section presents the behavioral ransomware detection methodology proposed in this paper and the evaluation metrics. The methodology consists of two main steps: Development and training of the ML models, and API call sequence monitoring and process classification.

4.1. Development and Training of the ML Models

To construct a robust and accurate ransomware detection system, we employed a diverse set of machine learning models, each selected for its ability to capture different characteristics of the API call sequence data. The training process was conducted using the primary dataset described in Section 3.1, where each sample represents program behavior as a fixed-length sequence of 100 API calls. All model configurations and hyperparameters are summarized in Table 4 to ensure reproducibility. Hyperparameters for models such as SVM and XGBoost were optimized using grid search, while the LSTM model was trained using early stopping and regularization to prevent overfitting.

The trained models include the following:

- **Random Forest (RF):** A tree-based ensemble learning method that excels in high-dimensional feature spaces and handles imbalanced datasets well. Empirically, the model was trained with 200 estimators and a maximum tree depth of 15. Class weights were computed to mitigate the impact of class imbalance, which is common in malware datasets.
- **Long Short-Term Memory (LSTM):** To capture temporal dependencies in API call sequences, we implemented an LSTM-based neural network. The input sequence is first transformed through an embedding layer that maps API identifiers to dense

vector representations, enabling the model to learn relationships between API calls based on their contextual usage. The architecture includes stacked LSTM layers with dropout regularization, followed by dense layers for classification.

- Support Vector Machine (SVM) with Non-Linear Kernel: Given the complex and non-linear boundaries between benign and malicious behavior, we utilized an SVM with a radial basis function (RBF) kernel. Input features were normalized, and hyperparameters such as the regularization parameter (C) and kernel coefficient (gamma) were tuned through grid search to optimize model performance.
- K-Nearest Neighbors (KNN): As a non-parametric method, KNN serves as a simple yet effective baseline by classifying samples based on the majority label among the k most similar training samples. We experimented with multiple values of k and distance metrics to identify optimal configurations.
- eXtreme Gradient Boosting (XGBoost): This gradient-boosting framework was included due to its proven performance on structured data and its ability to effectively model complex interactions between features. XGBoost's regularization parameters were tuned to balance bias and variance, which is critical in security applications.
- Linear Discriminant Analysis (LDA): A method of dimensionality reduction that projects data onto a lower-dimensional space in a way that maximizes class separability. LDA was used to convert the high-dimensional API call sequences to a more compact form, which improved computational efficiency and suppressed noise. While its linear boundaries constrained performance on complicated cases, LDA served as a baseline to compare the non-linear models.

Table 4. Hyperparameter configurations for all trained models. XGBoost parameters selected via GridSearchCV (3-fold, 96 combinations). LSTM trained with early stopping on validation accuracy (patience 15). Ensemble weights validated through a 56-configuration sweep.

Model	Configuration
KNN	$k = 4$; distance: Euclidean; input: StandardScaler-normalized
LDA	Default scikit-learn solver (svd); input: StandardScaler-normalized
Random Forest	n_estimators = 200, max_depth = 15, class_weight = 'balanced', random_state = 42; input: raw integer features (scale-invariant)
SVM (RBF)	kernel = 'rbf', C = 1.0, gamma = 'scale', probability = True, random_state = 42; input: StandardScaler-normalized
XGBoost	Selected via GridSearchCV (3-fold, 96 combinations): n_estimators = 200, max_depth = 7, learning_rate = 0.1, subsample = 0.8, colsample_bytree = 0.9, gamma = 0.1, eval_metric = 'logloss', random_state = 42; Search space: n_estimators $\in \{100, 200\}$, max_depth $\in \{3, 5, 7\}$, learning_rate $\in \{0.01, 0.1\}$, subsample $\in \{0.8, 0.9\}$, colsample_bytree $\in \{0.8, 0.9\}$, gamma $\in \{0, 0.1\}$; Input: raw integer features
LSTM	Embedding(vocab = 307, dim = 32) $\rightarrow$ LSTM(128, rec. dropout = 0.2) $\rightarrow$ BatchNorm $\rightarrow$ LSTM(64, rec. dropout = 0.2) $\rightarrow$ BatchNorm $\rightarrow$ LSTM(32, rec. dropout = 0.2) $\rightarrow$ BatchNorm $\rightarrow$ Dense(32, ReLU, L2 = 0.01) $\rightarrow$ Dense(1, Sigmoid) Optimizer: Adam (lr = 0.001, clipnorm = 1.0); batch size = 64; early stopping on val_accuracy (patience = 15) Input: raw integer sequences (embedding handles representation)
Ensemble	Step 1: XGBoost + SVM via VotingClassifier (voting = 'soft', equal weights) Step 2: Weighted blend with LSTM probability: $P_{\text{final}} = 0.2 \cdot P_{\text{LSTM}} + 0.8 \cdot P_{\text{XGB+SVM}}$ Classification threshold = 0.5 Optimal from 56-config sweep: LSTM weight = 0.3, threshold = 0.55 (accuracy 99.83%)

The selection of these models reflects a combination of sequence-based, kernel-based, and tree-based approaches, allowing the system to capture complementary aspects of ransomware behavior. To further enhance performance, we integrated all models into a soft-voting ensemble. Predictions from individual models were combined using weighted probabilities, where weights were assigned based on cross-validation performance. This ensemble approach improves robustness and generalization, particularly in detecting previously unseen ransomware samples, as demonstrated in the zero-day evaluation.

4.2. API Call Sequence Monitoring and Process Classification

The core of the proposed detection mechanism centers around observing the behavior of the running processes through their use of Windows API calls. Since ransomware activity is typically reflected in anomalous sequences of system-level calls—such as file encryption, registry modification, or process injection—the system monitors these API invocations to identify malicious intent at runtime.

Figure 2 shows the flowchart of the API call sequence monitoring and process classification of the proposed system. The first step involves getting the list of running processes and extracting the features for each process. The second step includes classification and response mechanism.

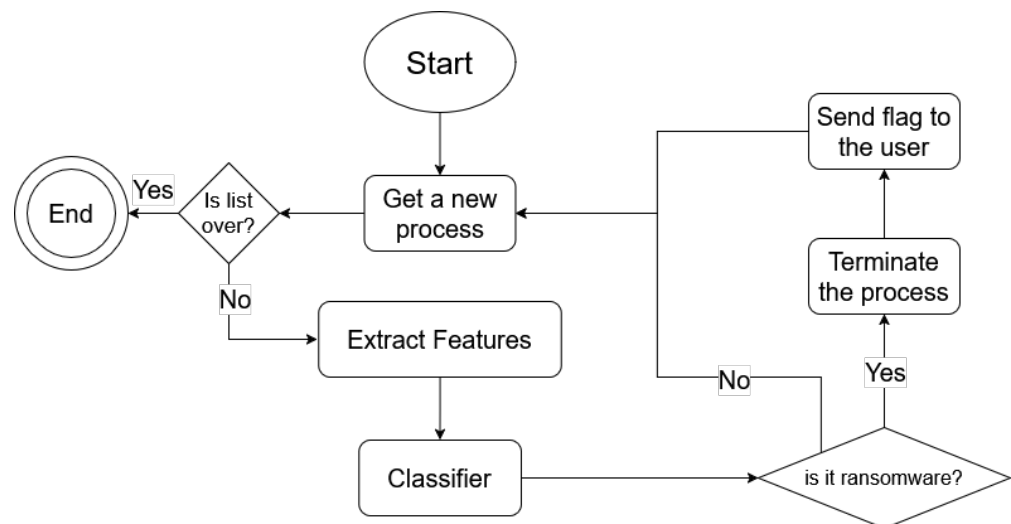


Figure 2. Flowchart showcasing an overview of the ransomware detection mechanism.

4.2.1. API Call Acquisition

The monitoring process continuously captures consecutive 100 API calls made by the selected running process. This is achieved through real-time dynamic monitoring using hook functions. A hook, as defined by Microsoft, “is a mechanism by which an application can intercept events such as messages, mouse actions, and keystrokes” [29]. In our case, the system is designed to intercept a sequence of 100 API calls (i.e., Win32 APIs) at the beginning of a process to establish an early behavioral profile. We hook all 307 Win32 APIs to log the Process ID (PID) that initiated the function call. This approach enables early-stage threat detection, allowing the system to respond before the ransomware completes its encryption or propagation routine. Unlike other detection techniques, which could be evaded by obfuscation, a malicious process will always have to interact with the operating system and underlying hardware of the computer using Windows API calls, as illustrated in Figure 3.

As shown in Figure 3, the monitoring agent continuously extracts the PIDs of currently running processes. Each PID is monitored until we have extracted 100 API calls. Once the required number of calls is observed, the sequence is passed to the classifier for evaluation.

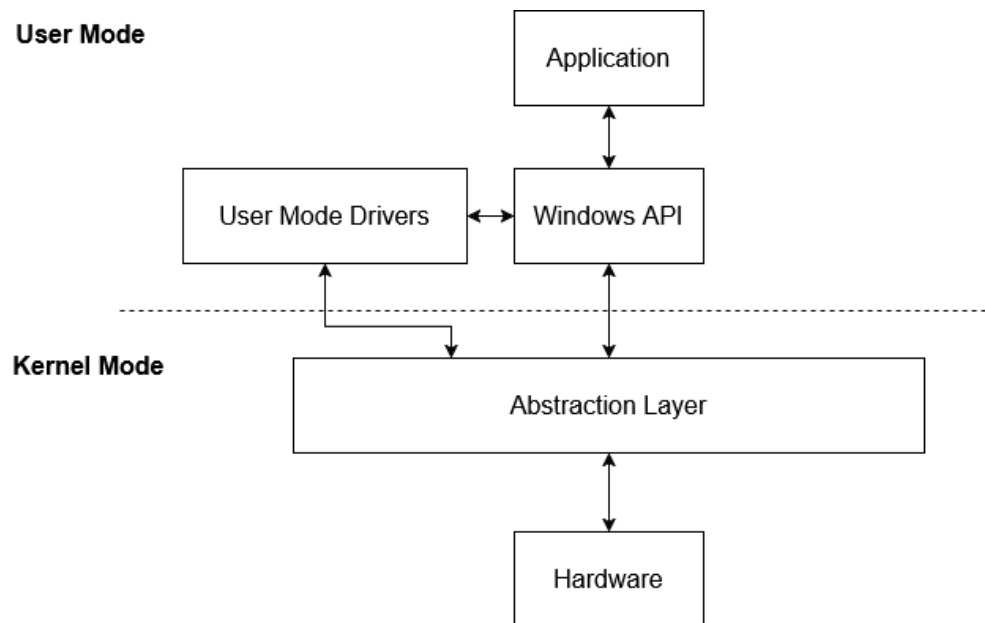


Figure 3. Windows operating modes.

We developed a monitoring script to collect the API call sequences for the samples in our zero-day dataset. In addition, the monitoring script is used to log the resource utilization, detection time and files saved. The script is based on the Frida Python library (version 17.9.5). Given a PID, the script would extract the first 100 Windows API calls made by the process. To parallelize this task, we used an OpenMP C++ script that would extract all the monitored processes on the computer and run the Python script for all PIDs in parallel. The algorithm for the API monitoring code is demonstrated in Algorithm 1.

Algorithm 1 API Monitoring Pseudocode

Require: Max calls $max_calls = 100$, Max time $max_time = 20$ s, Safe users $safe_users = [\dots]$

Ensure: Captures API calls from user processes

```

1: while not stopped do
2:   for each proc in running processes do
3:     if proc.owner  $\notin$  safe_users then
4:       continue
5:     end if
6:     Start timer  $t \leftarrow 0$ 
7:     Initialize empty call_list
8:     Attach hook to proc.pid
9:     while  $t < max\_time$  AND  $|call\_list| < max\_calls$  do
10:       $call \leftarrow \text{GetNextAPICall}(proc.pid)$ 
11:      Append call to call_list
12:    end while
13:    Detach hook
14:    Save call_list for proc.pid
15:   end for
16: end while

```

4.2.2. API Call Sequence Processing and Classification

As described in the previous subsection, each process is represented as an ordered sequence of API call identifiers corresponding to 100 API calls observed during execution. Every API is mapped to a unique integer index drawn from a fixed vocabulary of 307 monitored Win32 functions. The resulting sequence preserves the order in which calls occur, allowing the representation to reflect the progression of process behavior over time.

To reduce noise and avoid over-representing repetitive operations, consecutive redundant API calls are removed during preprocessing as mentioned in Section 3.1. As a result, the representation does not encode frequency information; instead, it captures the sequence of distinct behavioral events. This design emphasizes transitions between API calls rather than repeated invocation patterns, which aligns with the objective of modeling behavioral structure without introducing redundancy.

For traditional machine learning models, including SVM, KNN, LDA, Random Forest, and XGBoost, the sequence is used directly as a fixed-length vector of integer identifiers. The positional arrangement of these identifiers implicitly encodes local ordering patterns within the execution trace. SVM, KNN, and LDA operate on standardized versions of these vectors, while Random Forest and XGBoost use the raw integer inputs due to their scale-invariant nature. For the LSTM model, the integer-encoded sequence is first passed through an embedding layer that maps each API identifier to a dense vector representation. The embedding layer has a vocabulary size of 307 and an embedding dimension of 32. This transformation produces a sequence of continuous vectors that serve as input to the recurrent layers.

The embeddings are learned jointly with the model parameters during training, enabling the network to capture relationships between API calls based on their contextual usage. In this way, API calls that frequently appear in similar behavioral contexts are mapped to nearby regions in the embedding space, allowing the model to learn higher-level representations of process behavior beyond discrete identifiers. All model configurations and hyper-parameters are reported in Table 4.

If a process is deemed malicious, the system triggers a configurable response mechanism that may include terminating the process, blocking further execution, generating system alerts, and logging relevant metadata for forensic analysis. This behavioral approach to ransomware detection ensures that the system remains effective against both known and previously unseen threats, without relying on static signatures. By analyzing what the process does—rather than what it contains—the system is more resilient to obfuscation, polymorphism, and minor code modifications that typically hinder traditional antivirus methods.

4.3. Evaluation Metrics

To comprehensively evaluate the performance of the ransomware detection system, both classification accuracy and system efficiency were measured. The evaluation metrics were selected to assess the system's ability to detect threats reliably and respond in a timely manner, while maintaining minimal resource overhead.

4.3.1. Classification Metrics

The effectiveness of the detection system is quantified using standard binary classification metrics:

- True Positives (TP): The number of correctly identified ransomware instances.
- True Negatives (TN): The number of correctly identified benign (non-malicious) processes.
- False Positives (FP): The number of benign processes incorrectly flagged as ransomware.
- False Negatives (FN): The number of ransomware instances that were not detected by the system.

From these counts, the following derived metrics are computed:

- Accuracy: $(TP + TN) / (TP + TN + FP + FN)$;
- Precision: $TP / (TP + FP)$;
- Recall (Sensitivity): $TP / (TP + FN)$;
- F1-Score: $2 \times (Precision \times Recall) / (Precision + Recall)$.

Additionally, the False Positive Rate (FPR) and False Negative Rate (FNR) are computed to gain insight into specific areas of performance:

- $FPR = FP / (FP + TN)$;
- $FNR = FN / (FN + TP)$.

A low FPR ensures that legitimate user activity is not unnecessarily disrupted, while a low FNR ensures ransomware instances are effectively contained.

4.3.2. System Efficiency Metrics

To evaluate the system's viability for real-time deployment, resource utilization and operational responsiveness were carefully measured. These metrics assess not only the system's efficiency under realistic conditions but also its practical effectiveness in halting attacks before damage is done.

- **CPU Usage:** The CPU load during runtime was monitored to assess the processing demands introduced by real-time API monitoring and model inference.
- **Memory Consumption:** The system's memory footprint was tracked to ensure the detection mechanism is suitable for environments with limited resources.
- **Detection Latency:** Defined as the elapsed time between the start of a ransomware process and the moment it is flagged or interrupted by the system.
- **Ransomware Impact Assessment:** Monitoring the quantity and total size of files affected or encrypted by the malicious process before it is interrupted.

To monitor these metrics, we used a custom ransomware implementation to mimic real-world scenarios while also being able to collect data like the number of encrypted files. The pseudocode for the custom ransomware implementation is shown in Algorithm 2.

Algorithm 2 Custom Ransomware Pseudocode

Require: Target directory *dir*, AES key size *key_size* = 32, Chunk size *chunk_size* = 64 KB

- 1: Generate random key *key* $\leftarrow$ RandomBytes(*key_size*)
- 2: **for** each *file* in *dir* and subdirectories **do**
- 3: Initialize AES-CTR cipher with *key*
- 4: Generate random 16-byte *iv*
- 5: Open *file* in read-write binary mode
- 6: Write *iv* at file start
- 7: **for** each *chunk* in file (size *chunk_size*) **do**
- 8: Encrypt *chunk* in-place
- 9: **end for**
- 10: Close *file*
- 11: **end for**

Together, these system-level metrics allow us to evaluate the practicality of deploying the proposed model in real-world scenarios, ensuring not only high detection accuracy but also minimal collateral damage and system overhead.

5. Results and Discussion

The proposed ensemble model was evaluated using the primary dataset and the zero-day dataset and was compared with various standalone ML models. In addition, the performance of the ensemble model was evaluated based on detection time, percentage of files saved (i.e., not encrypted), CPU and memory utilization.

5.1. Primary Dataset Testing

In this section, we discuss the classification results of multiple ML models using the primary dataset. We implemented and evaluated several classification models to detect

ransomware based on API call sequences. The models tested include LSTM, KNN, XGBoost, LDA, RF, SVM with a non-linear kernel, and an ensemble voting classifier that aggregates predictions from the top three performing models (i.e., XGBoost, LSTM, and SVM) using soft-voting. Each algorithm was assessed using a comprehensive set of metrics, including precision, recall, and F1-score for both the benign and ransomware classes, alongside accuracy, FPR, FNR, Area Under the Receiver Operating Characteristic Curve (ROC-AUC), and 95% Wilson score confidence intervals (CI) separately for both benign (class 0) and ransomware (class 1) classes. We chose the Wilson score method for CIs because the normal approximation performs poorly when accuracy is near 1.0, which is the case for several of our models:

- RF: This model achieved balance between precision (0.99 and 1.0), recall (1.0 and 0.98), and low FNR (0.0215). It also provided robust feature importance estimates, which we leveraged for feature selection. With accuracy up to 99%, RF proved to be one of the most reliable standalone classifiers.
- LSTM: This model is chosen for its ability to capture temporal dependencies in sequential data. LSTM achieved an accuracy of 97%, with an impressive F1-score of 0.98 for ransomware. It proved particularly effective at detecting malicious behavior early in the API call stream, leveraging its recurrent structure to model call-order relationships.
- SVM (Non-linear Kernel): One of the highest-performing standalone models with an accuracy of 99.55%, SVM demonstrated excellent precision, recall, and F1-scores across both classes. Its FPR (0.0012) and FNR (0.0100) were among the lowest of all models, confirming its effectiveness in handling non-linear separability in API behavior.
- KNN: A simple but effective distance-based model. KNN performed well in recall (1.0) for benign cases but showed a slightly higher FNR of 0.0717. While it benefits from ease of implementation and interpretability, its performance degrades with larger datasets and high-dimensional spaces.
- XGBoost: This model is known for its regularized gradient boosting. XGBoost reached the highest accuracy (99.56%) and exhibited low FPR (0.0012) and FNR (0.0096), proving its high performance for structured data.
- LDA: This linear classifier model yielded the lowest performance among all models, with accuracy at 90.36% and high FNR (0.1426). Its simplistic linear boundaries were insufficient to capture the complexities and variability of API behavior in ransomware.
- Ensemble: To harness the strengths of multiple classifiers, a soft voting ensemble was implemented using LSTM, XGBoost, and SVM. The ensemble model achieved 99.82% accuracy with improved generalization and reduced variance. The FPR and FNR were 0.0005 and 0.0038, respectively—balancing the trade-off between false alarms and missed detection.

The ensemble model was particularly effective in scenarios where individual models exhibited complementary strengths. For instance, LSTM's temporal modeling, XGBoost's robustness to noise, and SVM's margin maximization collectively contributed to a more resilient detection mechanism.

The superior performance of the ensemble can be attributed to the complementary strengths of its constituent models. The LSTM captures temporal dependencies in API call sequences, XGBoost effectively models complex feature interactions in structured representations, and SVM provides strong non-linear decision boundaries. Combining these models through soft voting reduces variance and improves robustness compared to individual classifiers.

Although both Random Forest and XGBoost are tree-based methods, XGBoost consistently achieved higher performance during individual model evaluation and was therefore selected for inclusion in the ensemble, while RF was excluded to avoid redundancy.

This selection ensures that the ensemble integrates diverse learning paradigms, including sequence-based, tree-based, and kernel-based, rather than overlapping model behaviors.

A summary of all evaluation metrics is provided in Table 5, while the ROC curves for all models are shown in Figure 4. The ensemble achieves a ROC-AUC of 0.9999, and all models except LDA exceed 0.98, indicating that performance remains consistently high across classification thresholds. Additionally, 5-fold stratified cross-validation results for KNN, LDA, RF, and SVM are presented in Table 6. The low standard deviations (0.08–0.30%) confirm the stability of these models across different data splits. For the LSTM, a held-out validation set with early stopping (patience = 15) was used to control overfitting during training.

Table 5. Model performance on the primary test set ($n = 13,695$). Class 0 = Benign, Class 1 = Ransomware. 95% confidence intervals computed using the Wilson score method. ROC-AUC computed from predicted class probabilities. Ensemble composition: XGBoost + SVM + LSTM (LSTM weight = 0.2, threshold = 0.5).

Model	Accuracy	95% CI	Benign (Class 0)			Ransomware (Class 1)			FPR	FNR	ROC-AUC
			Prec.	Rec.	F1	Prec.	Rec.	F1			
KNN	0.9725	[96.97%, 97.52%]	0.96	1.00	0.98	1.00	0.93	0.96	0.0004	0.0717	0.9847
LDA	0.9036	[89.86%, 90.84%]	0.91	0.93	0.92	0.89	0.86	0.87	0.0681	0.1426	0.9476
Random Forest	0.9912	[98.95%, 99.26%]	0.99	1.00	0.99	1.00	0.98	0.99	0.0011	0.0215	0.9997
SVM (RBF)	0.9955	[99.42%, 99.65%]	0.99	1.00	1.00	1.00	0.99	0.99	0.0012	0.0100	0.9991
XGBoost	0.9956	[99.44%, 99.66%]	0.99	1.00	1.00	1.00	0.99	0.99	0.0012	0.0096	0.9998
LSTM	0.9893	[98.74%, 99.09%]	0.99	0.99	0.99	0.98	0.99	0.98	0.0090	0.0136	0.9985
Ensemble (XGB+SVM+LSTM)	0.9982	[99.74%, 99.88%]	1.00	1.00	1.00	1.00	1.00	1.00	0.0005	0.0038	0.9999

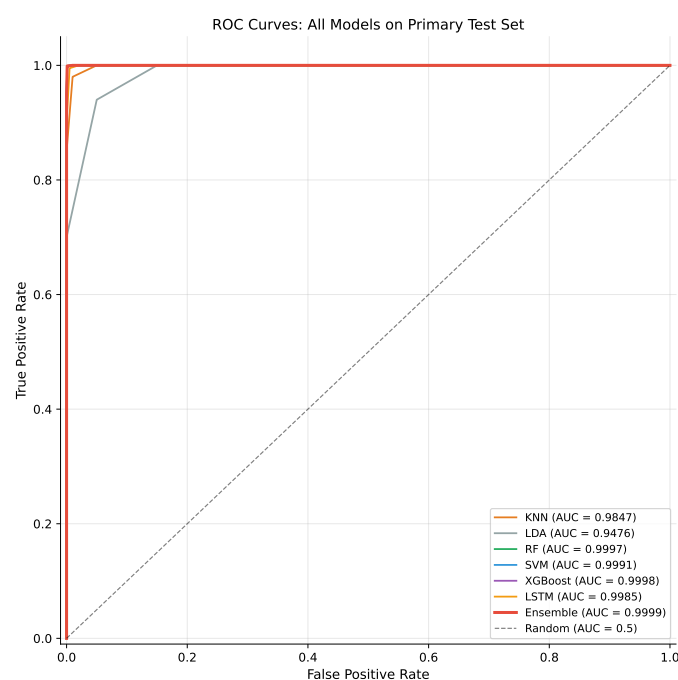


Figure 4. ROC curves for all evaluated models on the primary test set. The ensemble reaches ROC-AUC = 0.9999. All models except LDA (0.9476) exceed 0.98, confirming that the high accuracy results are stable across classification thresholds rather than being artifacts of a single threshold choice.

Table 6. 5-fold stratified cross-validation on the training set. LSTM is excluded due to Keras/scikit-learn incompatibility with `cross_val_score`; a held-out validation split with early stopping (patience 15) was used instead during LSTM training. Each model uses the same hyperparameters as for primary test evaluation.

Model	Mean Accuracy	Std Dev
KNN	96.94%	$\pm 0.20\%$
LDA	90.12%	$\pm 0.30\%$
Random Forest	98.97%	$\pm 0.08\%$
SVM (RBF)	99.37%	$\pm 0.08\%$
XGB	99.43%	$\pm 0.0008\%$

Although XGBoost demonstrated strong standalone performance (i.e., 99.56% accuracy) on our limited evaluation set, previous research [30] suggests that ensemble models often outperform single classifiers like XGBoost when applied to larger and more diverse datasets, particularly in zero-day scenarios. This is because ensemble models combine multiple perspectives, allowing them to capture a broader range of behavioral signals. Furthermore, when facing more advanced ransomware strains that attempt to evade detection by mimicking benign behavior or employing delayed malicious activity, the ensemble's ability to analyze a broader range of features becomes crucial for maintaining robust detection performance. Table 4 lists the hyperparameter configurations for all trained models. These parameters can be used to regenerate the results achieved in this paper.

5.2. Zero-Day Dataset Testing

To evaluate the robustness of our system in handling previously unseen threats, we tested all models on the zero-day dataset described in Section 3.2. As illustrated in Figure 5, our ensemble model achieved the highest accuracy (96.3%), outperforming SVM (91.3%) and XGBoost (88.1%). RF generated an accuracy of 78.9%, while LSTM exhibited lower performance at 74.3%. 95% Wilson score confidence intervals are illustrated as error bars in Figure 5. The ensemble (XGB + SVM + LSTM) achieves 96.33% (CI: 90.94–98.56%), the highest of all models.

KNN underperformed significantly, yielding a low accuracy of just 21.1%, highlighting its vulnerability to class imbalance and lack of generalization in high-dimensional, unseen data. Hence, KNN was excluded from our final ensemble. The high accuracy, precision, recall, and F1 performance of our combined ensemble model integrates predictions from both neural and tree-based architectures. This demonstrates the ensemble model's importance in detecting new ransomware strains (i.e., 109 zero-day testing samples).

5.3. Detection Time Analysis

To evaluate the system's responsiveness, we conducted 50 test runs using the custom ransomware sample (Section 4.3.2) and recorded the time taken by the ensemble model to detect and flag each malicious process. To test the detection method in a realistic environment we used Corpora [31]. Corpora is a dataset of 12.8 K office files (including CSV, PDF, DOCX, PPTX, XLSX, TXT, JPG, and more) of 11 GB in size. As shown in Figure 6, the detection time varied across runs due to differences in ransomware behavior and system load. Detection time ranged from as low as 2.5 s to a maximum of 38 s, with a mean detection time of 15.3 s and a standard deviation of 7.8 s.

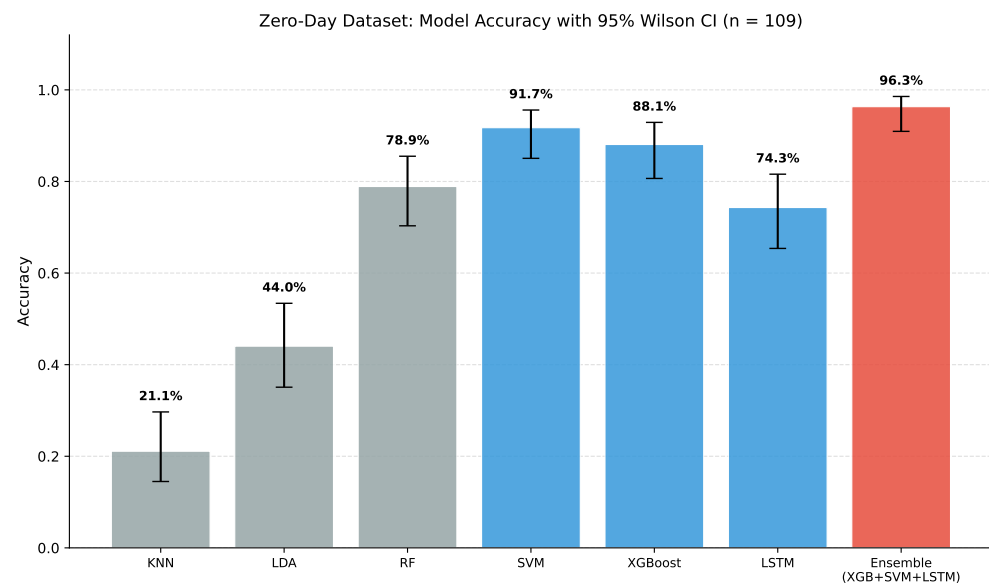


Figure 5. Model accuracy on the zero-day dataset ($n = 109$) with 95% Wilson score confidence intervals shown as error bars. The ensemble (XGB + SVM + LSTM) achieves 96.33% (CI: 90.94–98.56%), the highest of all models. Wide intervals reflect the small sample size. Even the ensemble's lower CI bound (90.94%) exceeds most individual model point estimates.

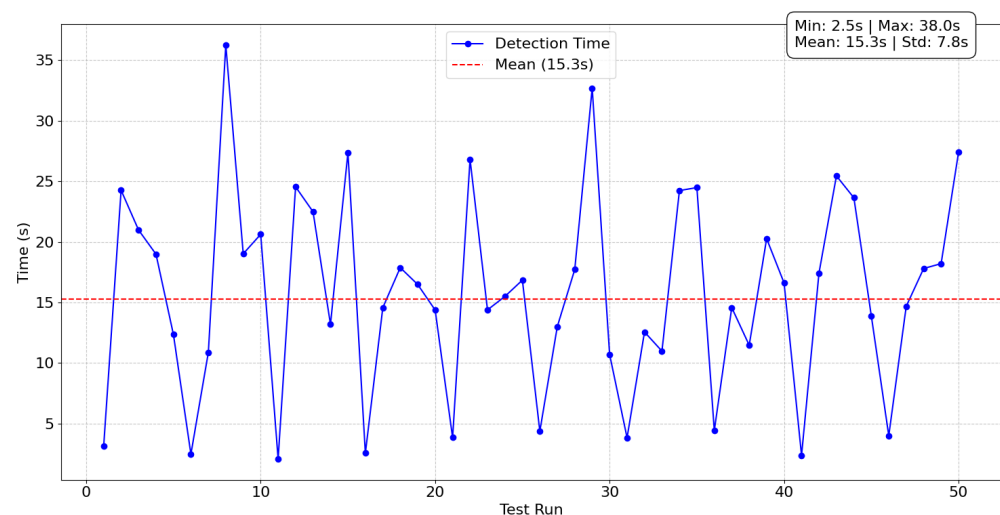


Figure 6. Detection time recorded across 50 test runs.

This performance demonstrates the system's ability to detect most threats well before significant damage is inflicted. The relatively low average detection time suggests that our real-time API call monitoring is effective under varied conditions. Occasional spikes in detection time correspond to stealthier ransomware strains or delayed execution of malicious routines, highlighting the need for early-stage behavioral profiling. The red dashed line in the chart marks the average detection time, serving as a benchmark for expected system latency in a production environment.

Encryption time for the same amount of data may vary depending on many factors like the CPU speed, RAM size, hard disk type, encryption type and other running processes. To quantify the impact of early detection on file integrity alongside the detection time, we conducted experiments to measure the percentage of files saved at varying data volumes. Similar to the previous test measuring the detection time, Corpora was used to measure the percentage of files saved [31]. Like any behavioral detection method, the proposed method

is vulnerable to file loss, as the ransomware has to start working to be detected. As a result, ransomware that uses fast encryption algorithms will increase file loss.

As shown in Figure 7, the detection system consistently prevented extensive encryption by terminating the ransomware process promptly. Across test environments containing 5 GB to 20 GB of target data, the system was able to preserve over 90% of files in all scenarios. The best preservation rate occurred at 15 GB, with 96% of files saved, demonstrating the effectiveness of our monitoring system. Even at the highest test volume (20 GB), the system maintained a 94% preservation rate, indicating strong scalability of the defense mechanism. These results confirm that the implemented models can successfully intervene during early stages of ransomware execution, significantly reducing data loss.

Overall, the experimental evaluation demonstrates the effectiveness of our proposed approach in detecting both known and previously unseen ransomware threats (limited to the 109 testing samples) working on different file variations and detection models. Through extensive benchmarking across multiple machine learning models and a custom ensemble, we demonstrated consistently high accuracy, precision, and recall scores, with the ensemble model offering a favorable balance between detection speed and reliability. Performance remained robust in real-world simulation tests, including zero-day scenarios, where the system achieved notable detection times and preserved a significant percentage of user files. These results validate the practicality of our framework for proactive ransomware mitigation in dynamic environments. However, we want to acknowledge that real-world live testing was done on 109 samples that are running within a sandbox. While testing on those 109 samples boosts our confidence in the dataset results, more testing might be needed to reach a more definite conclusion.

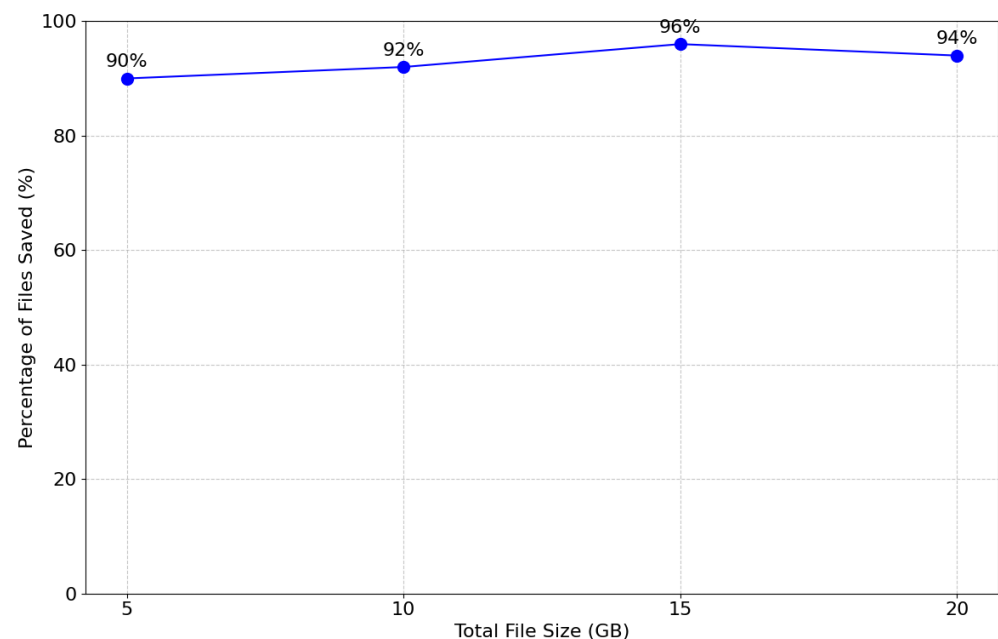


Figure 7. The percentage of files successfully saved in relation to the size of all files.

5.4. Resources Utilization

Throughout our benchmarking tests, we measured both CPU and memory utilization at regular intervals to evaluate the resource impact of the proposed detection system, as shown in Figure 8.

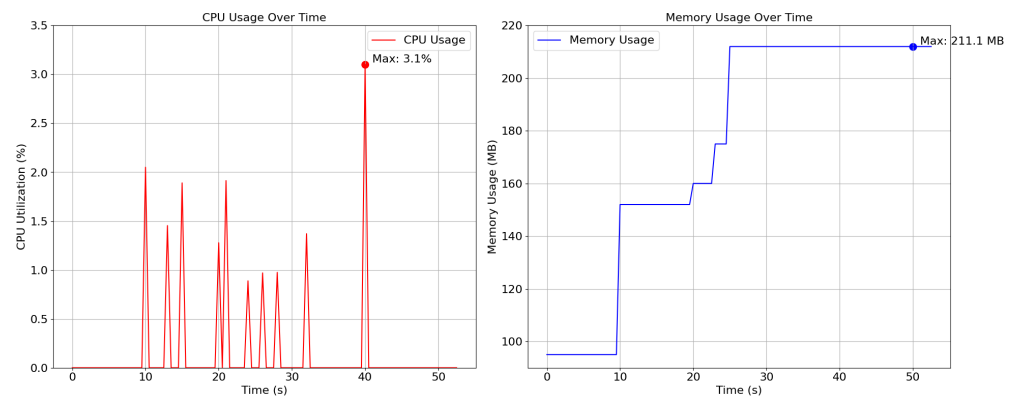


Figure 8. CPU and memory usage trends observed during real-time ransomware detection.

The results demonstrate minimal system resource impact, with CPU usage remaining stable throughout the test and reaching a maximum of 3.1%. Memory consumption peaked at 211.1 MB. Importantly, these measurements were taken while ransomware was executing concurrently with the proposed detection system, to simulate real-world deployment conditions.

6. Conclusions and Future Work

In summary, this paper presented a behavior-based ransomware detection system leveraging dynamic API call monitoring and ensemble machine learning to identify and mitigate threats in real-time. We developed a modular framework combining LSTM networks for temporal pattern recognition, XGBoost for structural robustness, and SVM for high-precision classification, integrated through a voting ensemble to maximize detection accuracy while minimizing false positives.

Our results demonstrate the effectiveness of this approach, with the ensemble achieving 99.82% accuracy and precision (1.0) across both benign and malicious samples. Notably, the system detected zero-day ransomware with 96.3% accuracy and preserved 90–96% of files from encryption, all while maintaining low resource overhead (CPU $\leq 3.1\%$, RAM 211 MB). These findings underscore the value of multi-model collaboration for ransomware detection, where each technique contributes unique strengths: LSTM captures sequential dependencies, SVM excels in margin maximization, and XGBoost ensures high performance on structured data and demonstrates robustness to noise.

Future enhancements could integrate network traffic analysis to identify exfiltration or C2 communication, as well as adaptive thresholds to minimize false positives during legitimate file operations. Advanced ML optimization methods (e.g., Bayesian) can be used in future work to provide stronger guarantees for model generalization. Privacy considerations must also be addressed, as API monitoring requires transparent user consent and safeguards against data misuse. Finally, a hybrid approach combining our behavioral model with static signatures or anomaly detection could further improve coverage, bridging the gap between research and industry-ready solutions.

Author Contributions: Conceptualization, A.A. and W.D.; methodology, A.A., W.D., Y.A. and O.A.; software, Y.A., O.A., O.T. and M.A.; validation, A.A., W.D., Y.A. and O.A.; formal analysis, A.A., W.D., Y.A., O.A., O.T. and M.A.; investigation, O.T. and M.A.; resources, A.A.; data curation, Y.A., O.A., O.T. and M.A.; writing—original draft preparation, A.A., W.D., Y.A., O.A., O.T. and M.A.; writing—review and editing, A.A. and W.D.; visualization, Y.A., O.A., O.T. and M.A.; supervision, A.A. and W.D.; project administration, A.A.; funding acquisition, A.A. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported in part by the American University of Sharjah under Grant number UGFY25-3152-UE2510. In addition, this work has been carried out during sabbatical leave granted to Waleed Dweik from the University of Jordan during the academic year 2024/2025.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The original contributions presented in the study are included in the article; further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

RF	Random Forest
LSTM	Long Short-Term Memory
SVM	Support Vector Machine
KNN	K-Nearest Neighbors
LD	Linear Discrimination
XGBoost	eXtreme Gradient Boosting
FNR	False Negative Rate
FPR	False Positive Rate
PE	Portable Executable

References

1. What Was the WannaCry Ransomware Attack? Available online: <https://www.cloudflare.com/learning/security/ransomware/wannacry-ransomware/> (accessed on 3 May 2026).
2. Win32/Locky. Available online: <https://www.microsoft.com/en-us/wdsi/threats/malware-encyclopedia-description?Name=Win32/Locky> (accessed on 29 May 2025).
3. Almashhadani, A.O.; Kaiiali, M.; Sezer, S.; O’Kane, P. A multi-classifier network-based crypto ransomware detection system: A case study of locky ransomware. *IEEE Access* **2019**, *7*, 47053–47067. [CrossRef]
4. Cen, M.; Jiang, F.; Qin, X.; Jiang, Q.; Doss, R. Ransomware early detection: A survey. *Comput. Netw.* **2024**, *239*, 110138. [CrossRef]
5. Oz, H.; Aris, A.; Levi, A.; Uluagac, A.S. A survey on ransomware: Evolution, taxonomy, and defense solutions. *ACM Comput. Surv. (CSUR)* **2022**, *54*, 1–37. [CrossRef]
6. Hassan, N.A. Ransomware Families: The Most Prominent Ransomware Strains. In *Ransomware Revealed: A Beginner’s Guide to Protecting and Recovering from Ransomware Attacks*; Apress: Berkeley, CA, USA, 2019; pp. 47–68.
7. Cicala, F.; Bertino, E. Analysis of encryption key generation in modern crypto ransomware. *IEEE Trans. Dependable Secur. Comput.* **2020**, *19*, 1239–1253. [CrossRef]
8. Fayi, S.Y.A. What Petya/NotPetya ransomware is and what its remidiations are. In *Proceedings of the Information Technology-New Generations: 15th International Conference on Information Technology*; Springer: Las Vegas, NV, USA, 2018; pp. 93–100.
9. Hsiao, S.C.; Kao, D.Y. The static analysis of WannaCry ransomware. In *Proceedings of the 2018 20th International Conference on Advanced Communication Technology (ICACT)*; IEEE: Chuncheon, Republic of Korea, 2018; pp. 153–158.
10. Usharani, S.; Bala, P.M.; Mary, M.M.J. Dynamic analysis on crypto-ransomware by using machine learning: Gandcrab ransomware. *Proc. J. Phys. Conf. Ser.* **2021**, *1717*, 012024. [CrossRef]
11. Cusack, G.; Michel, O.; Keller, E. Machine learning-based detection of ransomware using SDN. In *Proceedings of the 2018 ACM International Workshop on Security in Software Defined Networks & Network Function Virtualization*, Tempe, AZ, USA, 21 March 2018; pp. 1–6.
12. Chen, P.H.; Bodak, R.; Gandhi, N.S. Ransomware recovery and imaging operations: Lessons learned and planning considerations. *J. Digit. Imaging* **2021**, *34*, 731–740. [CrossRef] [PubMed]
13. Almousa, M.; Basavaraju, S.; Anwar, M. API-Based Ransomware Detection Using Machine Learning-Based Threat Detection Models. In *Proceedings of the 2021 18th International Conference on Privacy, Security and Trust (PST)*, Auckland, New Zealand, 13–15 December 2021; pp. 1–7. [CrossRef]

14. Naik, N.; Jenkins, P.; Gillett, J.; Mouratidis, H.; Naik, K.; Song, J. Lockout-Tagout Ransomware: A Detection Method for Ransomware using Fuzzy Hashing and Clustering. In Proceedings of the 2019 IEEE Symposium Series on Computational Intelligence (SSCI), Xiamen, China, 6–9 December 2019. [\[CrossRef\]](#)
15. Ayub, M.A.; Siraj, A.; Filar, B.; Gupta, M. RWArmor: A Static-Informed Dynamic Analysis Approach for Early Detection of Cryptographic Windows Ransomware. *Int. J. Inf. Secur.* **2024**, *23*, 533–556. [\[CrossRef\]](#)
16. Chen, Q.; Islam, S.R.; Haswell, H.; Bridges, R.A. Automated ransomware behavior analysis: Pattern extraction and early detection. In *Proceedings of the Science of Cyber Security: Second International Conference, SciSec 2019, Nanjing, China, 9–11 August 2019; Revised Selected Papers 2*; Springer: Nanjing, China, 2019; pp. 199–214.
17. Hirano, M.; Kobayashi, R. Machine Learning-based Ransomware Detection Using Low-level Memory Access Patterns Obtained From Live-forensic Hypervisor. In Proceedings of the 2022 IEEE International Conference on Cyber Security and Resilience (CSR), Virtual Conference, 27–29 July 2022; pp. 323–330. [\[CrossRef\]](#)
18. Kok, S.; Abdullah, A.; Jhanjhi, N. Early detection of crypto-ransomware using pre-encryption detection algorithm. *J. King Saud Univ. Comput. Inf. Sci.* **2022**, *34*, 1984–1999. [\[CrossRef\]](#)
19. Mehnaz, S.; Mudgerikar, A.; Bertino, E. RWGuard: A Real-Time Detection System Against Cryptographic Ransomware. In *Proceedings of the Research in Attacks, Intrusions, and Defenses*; Springer: Heraklion, Crete, Greece, 2018; pp. 114–136.
20. Bae, S.I.; Lee, G.B.; Im, E.G. Ransomware detection using machine learning algorithms. *Concurr. Comput. Pract. Exp.* **2020**, *32*, e5422. [\[CrossRef\]](#)
21. Cen, M.; Jiang, F.; Doss, R. RansoGuard: A RNN-based framework leveraging pre-attack sensitive APIs for early ransomware detection. *Comput. Secur.* **2025**, *150*, 104293. [\[CrossRef\]](#)
22. AlMajali, A.; Qaffaf, A.; Alkayid, N.; Wadhawan, Y. Crypto-ransomware detection using selective hashing. In *Proceedings of the 2022 International Conference on Electrical and Computing Technologies and Applications (ICECTA)*; IEEE: Ras Al Khaimah, United Arab Emirates, 2022; pp. 328–331.
23. AlMajali, A.; Elmosalamy, A.; Safwat, O.; Aboueila, H. Adaptive Ransomware Detection Using Similarity-Preserving Hashing. *Appl. Sci.* **2024**, *14*, 9548. [\[CrossRef\]](#)
24. Cen, M.; Deng, X.; Jiang, F.; Doss, R. Zero-Ran Sniff: A zero-day ransomware early detection method based on zero-shot learning. *Comput. Secur.* **2024**, *142*, 103849. [\[CrossRef\]](#)
25. Oliveira, A.; Sassi, R.J. Behavioral Malware Detection Using Deep Graph Convolutional Neural Networks. *TechRxiv* **2019**, Preprint. [\[CrossRef\]](#)
26. Raghuramu, A.; Pathak, P.H.; Zang, H.; Han, J.; Liu, C.; Chuah, C.N. Uncovering the footprints of malicious traffic in wireless/mobile networks. *Comput. Commun.* **2016**, *95*, 95–107. [\[CrossRef\]](#)
27. Li, C.; Lv, Q.; Li, N.; Wang, Y.; Sun, D.; Qiao, Y. A Novel Deep Framework for Dynamic Malware Detection Based on API Sequence Intrinsic Features. *Comput. Secur.* **2022**, *116*, 102686. [\[CrossRef\]](#)
28. VirusShare. Available online: <https://virusshare.com/> (accessed on 5 July 2025).
29. Karl-Bridge-Microsoft. Hooks Overview—Win32 apps, n.d. Available online: https://learn.microsoft.com/en-us/windows/win32/winmsg/about-hooks#wh_callwndproc-and-wh_callwndprocret (accessed on 19 May 2026).
30. Igugu, A. Evaluating the Effectiveness of AI and Machine Learning Techniques for Zero-Day Attacks Detection in Cloud Environments, 2024. Available online: <https://www.diva-portal.org/smash/get/diva2:1890285/FULLTEXT02> (accessed on 19 May 2026).
31. Garfinkel, S.; Farrell, P.; Roussev, V.; Dinolt, G. Bringing science to digital forensics with standardized forensic corpora. *Digit. Investig.* **2009**, *6*, S2–S11. [\[CrossRef\]](#)

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.